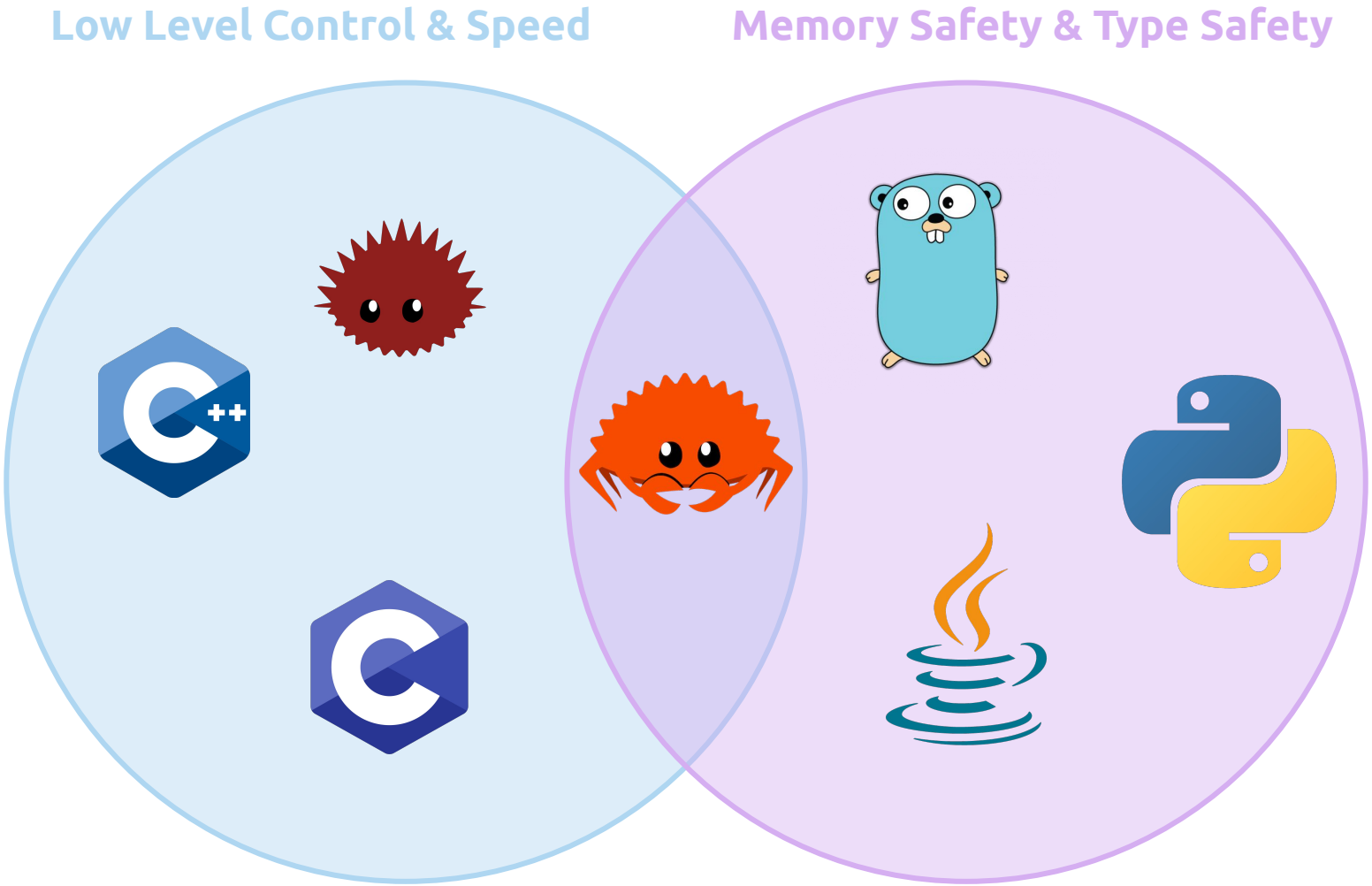
RustBelt: Securing the Foundations of the Rust Programming Language

A paper by Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, Derek Dreyer

A presentation by Jake Roggenbuck


Low Level Control & Speed
Memory Safety & Type Safety

What this paper includes

- **Formal** machine-checked safety proof for a subset of Rust
- Authors says this is the first for a **realistic subset** of Rust
- **Verification** conditions for the most common libraries in Rust
- A good intro to why **Rust** is so cool

Unsafe libraries as an extension of the language

Given a new unsafe library, is this a safe **extension** of the language?

The authors say they can find what condition the library needs to satisfy for it to be a **safe** addition to the language.

This could help developers test their libraries and other developers **audit libraries** that they are using.

The authors then checked the most common libraries.

Great Rust Features

- Ownership
  - **One** mutable reference to the memory
- Zero cost abstractions

How does one model Rust?

λ_{Rust}

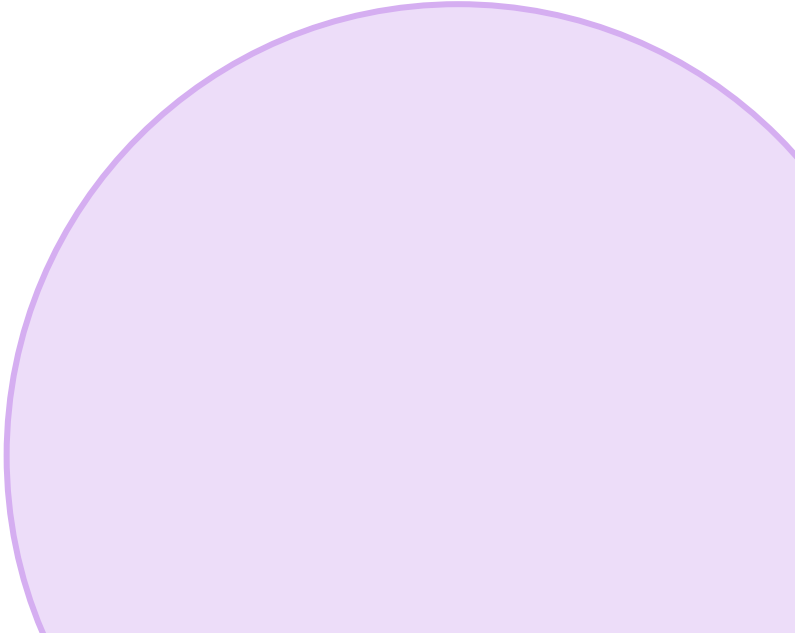
Formal version of Rust's type system

The problems to solve to model Rust

- “Choosing the right **logic** for modeling Rust”
- “Modeling Rust’s extensible **ownership** discipline”
- “Accounting for Rust’s “**thread-safe**” type bounds”

“Tour of Rust”

- Ownership Transfer
- Mutable References
- Shared References
- Lifetimes
- Interior Mutability



Ownership Transfer

```
let (snd, rcv) = channel();
join(move || {
        let mut v = Vec::new(); v.push(0); // v: Vec<i32>
        snd.send(v);
        // Cannot access v: v.push(1) rejected
      },
      move || {
        let v = rcv.recv().unwrap(); // v: Vec<i32>
        println!("Received: {:?}", v);
      });
```

Mutable References

```rust
fn main() {
    let mut x = &mut 5;

    do_work(x);

    println!("{}", x);
}

fn do_work(n: &mut u32) {
    *n += 5;
}
```

https://stackoverflow.com/questions/40921712/rust-mutable-value-vs-mutable-reference

Shared References

```
1  let mut v = Vec::new(); v.push(1);
2  join(|| println!("Thread 1: {:?}", &v), || println!("Thread 2: {:?}", &v));
3  v.push(2);
```

Lifetimes

```
1  fn example(v: &/* 'a */mut Vec<i32>) {
2    v.push(21);                    Lifetime 'c
3    { let mut head : &/* 'b */mut i32 = v.index_mut(0);
4      // Cannot access v: v.push(2) rejected
5      *head = 23; }                                Lifetime 'b
6    v.push(42);
7    println!("{:?}", v); // Prints [23, ..., 42]
8  }                                                Lifetime 'a
```

Interior Mutability

2.5.1 *Cell.* The simplest type with interior mutability is `Cell`. Consider the following example:

```
let c1 : &Cell <i32> = &Cell::new(0);
let c2 : &Cell <i32> = c1;
c1.set(2);
println!("{:?}", c2.get()); // Prints 2
```

[3] **usize** is an unsigned integer type of platform-dependent size large enough to cover the address space.

Other e.g. Mutex

Syntax

$$
\begin{aligned}
\mathit{Path} \ni p &::= x \mid p.n \\
\mathit{Val} \ni v &::= \texttt{false} \mid \texttt{true} \mid z \mid \ell \mid \texttt{funrec}\, f(\overline{x})\, \texttt{ret}\, k := F \\
\mathit{Instr} \ni I &::= v \mid p \mid p_1 + p_2 \mid p_1 - p_2 \mid p_1 \leq p_2 \mid p_1 = p_2 \mid \texttt{new}(n) \mid \texttt{delete}(n, p) \\
&\quad \mid {}^{*}p \mid p_1 := p_2 \mid p_1 :=_n {}^{*}p_2 \mid p :\overset{\text{inj } i}{=} () \mid p_1 :\overset{\text{inj } i}{=} p_2 \mid p_1 :\overset{\text{inj } i}{=}_n {}^{*}p_2 \mid \ldots \\
\mathit{FuncBody} \ni F &::= \texttt{let}\, x = I\, \texttt{in}\, F \mid \texttt{letcont}\, k(\overline{x}) := F_1\, \texttt{in}\, F_2 \mid \texttt{newlft}; F \mid \texttt{endlft}; F \\
&\quad \mid \texttt{if}\, p\, \texttt{then}\, F_1\, \texttt{else}\, F_2 \mid \texttt{case}\, {}^{*}p\, \texttt{of}\, \overline{F} \mid \texttt{jump}\, k(\overline{x}) \mid \texttt{call}\, f(\overline{x})\, \texttt{ret}\, k
\end{aligned}
$$

Syntax

```
fn option_as_mut<'a>
    (x: &'a mut Option<i32>) ->
    Option<&'a mut i32> {
  match *x {
    None => None,
    Some(ref mut t) => Some(t)
  }
}
```

$$
\begin{array}{l}
\textbf{funrec}\ \mathtt{option_as_mut}(\mathtt{x})\ \textbf{ret}\ \mathtt{ret} := \\
\quad \textbf{let}\ \mathtt{r} = \textbf{new}(2)\ \textbf{in} \\
\quad \textbf{letcont}\ \mathtt{k}() := \textbf{delete}(1, \mathtt{x}); \textbf{jump}\ \mathtt{ret}(\mathtt{r})\ \textbf{in} \\
\quad \textbf{let}\ \mathtt{y} = {}^{*}\mathtt{x}\ \textbf{in}\ \textbf{case}\ {}^{*}\mathtt{y}\ \textbf{of} \\
\quad - \ \mathtt{r} :\overset{\mathrm{inj}\,0}{=} (); \textbf{jump}\ \mathtt{k}() \\
\quad - \ \mathtt{r} :\overset{\mathrm{inj}\,1}{=} \mathtt{y}.1; \textbf{jump}\ \mathtt{k}()
\end{array}
$$

Type System

$$\mathit{Lft} \ni \kappa ::= \alpha \mid \textbf{static} \qquad \mathbf{E} ::= \emptyset \mid \mathbf{E}, \kappa \sqsubseteq_{\mathrm{e}} \kappa' \qquad \mathbf{T} ::= \emptyset \mid \mathbf{T}, p \lhd \tau \mid \mathbf{T}, p \lhd^{\dagger\kappa} \tau$$

$$\mathit{Mod} \ni \mu ::= \textbf{mut} \mid \textbf{shr} \qquad \mathbf{L} ::= \emptyset \mid \mathbf{L}, \kappa \sqsubseteq_{\mathrm{l}} \overline{\kappa} \qquad \mathbf{K} ::= \emptyset \mid \mathbf{K}, k \lhd \textbf{cont}(\mathbf{L}; \overline{x}.\, \mathbf{T})$$

$$\mathit{Type} \ni \tau ::= T \mid \textbf{bool} \mid \textbf{int} \mid \textbf{own}_n\, \tau \mid \&^{\kappa}_{\mu}\, \tau \mid \lightning_n \mid \Pi\overline{\tau} \mid \Sigma\overline{\tau} \mid \forall\overline{\alpha}.\, \textbf{fn}(\mathfrak{f} : \mathbf{E}; \overline{\tau}) \to \tau \mid \mu\, T.\, \tau$$